Express.js Routing –

1. Introduction to Routing in Express.js

Routing is one of the core concepts in Express.js. In simple terms, a **route represents a URL**. When a user visits a URL, Express.js decides **which code should run** based on that route.

What is a Route?

- A route is the part of a URL that comes **after the domain name**.
- Example:

```
https://example.com/about
```

  Here, `/about` is the **route**.

Why Routing is Important

- Routes determine which page or data the server should send.
- Routes are used for:
  - Websites
  - APIs
  - Mobile applications
  - Frontend frameworks like React or Vue.js

---

2. Traditional URLs vs Express.js Routes

Traditional HTML-Based URLs

- URLs often expose:
  - File names
  - Folder structures
  - File extensions (`.html`, `.php`)
- Example:

```
example.com/pages/about.html
```

- Disadvantages:
  - Security risk (exposes internal structure)

- Tight coupling with file extensions

Express.js Routes

- No file extensions in URLs
- Clean and secure URLs
- Example:

```
example.com/about
```

- Backend logic can map `/about` to any internal file

---

3. Types of Routes in Express.js

3.1 Basic Routes

- `/` → Home page
- `/about` → About page
- `/gallery` → Gallery page

3.2 Nested (Sub) Routes

Routes can be nested to represent hierarchy.

Example:

```
/about/user
```

---

4. HTTP Requests and CRUD Operations

Express.js handles **HTTP requests**, which map directly to **CRUD operations**:

Operation	HTTP Method	Purpose
Read	GET	Fetch data or view a page
Create	POST	Add new data
Update	PUT	Modify existing data
Delete	DELETE	Remove data

Express provides **methods** for each:

- `app.get()`

- `app.post()`
- `app.put()`
- `app.delete()`

---

5. Express.js Project Setup (Recap)

Installed Packages

- **express** – main framework
- **nodemon** – auto-restarts server on file changes

Basic Server Setup

```
const express = require('express');
const app = express();

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

---

6. Creating Routes Using GET Method

Home Route

```
app.get('/', (req, res) => {
  res.send('<h1>Welcome to Home Page</h1>');
});
```

Access in browser:

```
http://localhost:3000/
```

About Route

```
app.get('/about', (req, res) => {
  res.send('<h1>About Page</h1>');
});
```

Access:

```
http://localhost:3000/about
```

Gallery Route

```
app.get('/gallery', (req, res) => {
  res.send('<h1>Gallery Page</h1>');
});
```

---

7. Nested Routes (Sub-Routes)

Example:

```
app.get('/about/user', (req, res) => {
  res.send('<h1>User Page</h1>');
});
```

Access:

```
http://localhost:3000/about/user
```

---

8. Handling Invalid Routes

If a route does not exist, Express returns an error:

```
Cannot GET /about-us
```

This happens when the route is not defined in code.

---

9. Special Characters in Routes

Express routes can contain special characters:

- `.`
- `@`
- `$`
- `-`

Example:

```
app.get('/random.text', (req, res) => {
  res.send('Random Page');
});
```

Access:

```
http://localhost:3000/random.text
```

---

10. Route Parameters (URL Parameters)

Route parameters allow sending **dynamic values** via the URL.

Single Route Parameter

```
app.get('/about/:id', (req, res) => {
  res.send(req.params);
});
```

Access:

```
/about/12
```

Output (JSON):

```
{
  "id": "12"
}
```

Multiple Route Parameters

```
app.get('/user/:userId/book/:bookId', (req, res) => {
  res.send(req.params);
});
```

Access:

```
/user/10/book/22
```

Output:

```
{
  "userId": "10",
  "bookId": "22"
}
```

Accessing Individual Route Parameters

```
res.send(req.params.userId);
```

or

```
res.send(req.params.bookId);
```

Sending Route Parameters with Messages

```
res.send("Book ID: " + req.params.bookId);
```

Multiple Parameters in a Single Route

```
app.get('/user/:userId-:bookId', (req, res) => {
  res.send(req.params);
});
```

Access:

```
/user/john123-22
```

Output:

```
{
  "userId": "john123",
```

```
  "bookId": "22"
}
```

---

11. Query Parameters

Query parameters are sent using a **question mark (?)**.

Example URL:

```
/search?name=john123
```

Accessing Query Parameters

```
app.get('/search', (req, res) => {
  res.send(req.query);
});
```

Output:

```
{
  "name": "john123"
}
```

Multiple Query Parameters

```
/search?name=john123&age=20&city=Goa
```

Output:

```
{
  "name": "john123",
  "age": "20",
  "city": "Goa"
}
```

Extracting Specific Query Values

```
app.get('/search', (req, res) => {
  const name = req.query.name;
  const age = req.query.age;

  res.send(`Name: ${name}, Age: ${age}`);
});
```

---

12. Summary of Key Concepts

- Routes define how URLs map to server logic
- Express supports:
  - Basic routes
  - Nested routes
  - Route parameters
  - Query parameters
- HTTP methods map to CRUD operations
- `req.params` → Route parameters
- `req.query` → Query parameters
- `res.send()` → Sends response to the client
